

rest-sample-dubbo-consumer Kullanıcı Rehberi

Bu rehber ilk kullanım içindir.

Amaç kısa ve nettir: Dubbo provider'daki veriyi Rust-Java REST API olarak dışarı açmak.

İçindekiler

1. Bu Proje Ne İşe Yarar?
2. Akış Nasıl Çalışır?
3. Build Sırasında Ne Üretilir?
4. Ne Zaman Kullanılır?
5. Gerçek Hayat Senaryoları
6. Hızlı Başlangıç
7. Kopyala-Yapıştır API Örnekleri
8. Endpoint'ler
9. Profile Seçimi
10. DB Pool Ve c64/c256 Ne Demek?
11. ZooKeeper Mi Static Service DNS Mi?
12. Sık Hatalar

Bu Proje Ne İşe Yarar?

`rest-sample-dubbo-consumer`, REST API açan sample uygulamadır.

REST handler Java tarafındadır. HTTP I/O Rust tarafındadır. Dubbo data path de minimum overhead için Rust tarafına yaklaştırılmıştır.

Consumer DB'ye doğrudan bağlanmaz. DB işi provider tarafındadır.

Akış Nasıl Çalışır?

```
flowchart LR
    A["Client"] --> B["Rust-Java REST Consumer"]
    B --> C["Java Handler"]
    C --> D["java-rust-dubbo"]
    D --> E["Dubbo Provider"]
    E --> F["PostgreSQL veya hazır JSON"]
    F --> E --> D --> C --> B --> A
```

Consumer tarafı ince kalmalıdır. Ağır DB ve mutation işleri provider sorumluluğudur.

Build Sırasında Ne Üretilir?

Dubbo client wrapper'ları elle yazılmaz. `@GenerateNativeDubboClient`, interface'e göre doğrudan client metotları üretir. Processor yalnızca Maven build sırasında çalışır. Interface metotları client oluşturulurken bir kez çözülür. Request sırasında proxy veya reflection ile metot aranmaz.

Maven build ayrıca `META-INF/reactor/components.idx` ve `META-INF/reactor/routes.idx` dosyalarını üretir. Framework startup sırasında bu listeleri kullanır. Geniş classpath taraması gerekmez. Index dosyalarını elle düzenlemeyin. Route çıkışması varsa build hata verir.

Ne Zaman Kullanılır?

Senaryo	Bu proje uygun mu?	Neden
REST API ile Dubbo provider dışarı açılacak	Evet	Ana kullanım budur.
Consumer DB'ye bağlanacak	Hayır	DB connection provider'da kalmalıdır.
En düşük memory isteniyor	Evet	<code>micro-dubbo</code> ve static provider DNS ile kullanın.
ZooKeeper zorunlu	Evet	<code>zookeeper-discovery</code> profile ile çalışır.

Senaryo	Bu proje uygun mu?	Neden
Provider hazır JSON dönüyor	Evet	En verimli kullanım budur.

Gerçek Hayat Senaryoları

Senaryo	Akış	Önerilen seçim
Katalog read API	Consumer provider'dan hazır JSON alır ve döner.	<code>native-static-consumer</code> + <code>micro-dubbo</code>
Müşteri listeleme	Consumer provider'a DB-backed read çağrısı yapar.	<code>micro-dubbo</code> , küçük route budget
Customer create	Request body provider command method'una gider.	<code>byte[]</code> command, idempotent <code>requestId</code>
Segment patch	Aynı customer için paralel update sınırlandırılır.	<code>customer-key-admission=1</code>
K8s ortamı	Provider adresi Service DNS ile bilinir.	Static provider DNS
ZooKeeper zorunlu kurum	Provider registry üzerinden bulunur.	<code>zookeeper-discovery</code> profile

Bu uygulamada consumer ince kalmalıdır.

DB connection, transaction ve mutation kuralı provider tarafında kalır.

Hızlı Başlangıç

Önce provider'ı başlatın.

Sonra consumer'ı çalıştırın:

```
mvn -q package
java -jar target/rest-sample-dubbo-consumer-0.4.0.jar
```

Health:

```
curl http://127.0.0.1:8080/app/health
```

DB read örneği:

```
curl http://127.0.0.1:8080/api/v1/customers/db
```

Customer create örneği:

```
curl -X POST http://127.0.0.1:8080/api/v1/customers `
-H "Content-Type: application/json" `
-d '{"requestId":"req-1","customerNo":"CUST-1001","fullName":"Ayşe Demir","segment":
"standard","email":"ayse@example.com"}'
```

Kopyala-Yapıştır API Örnekleri

Hazır catalog JSON oku:

```
curl http://127.0.0.1:8080/api/v1/catalog/nested
```

DB-backed customer listesi oku:

```
curl http://127.0.0.1:8080/api/v1/customers/db
```

Tek customer oku:

```
curl http://127.0.0.1:8080/api/v1/customers/db/1
```

Segment bazlı customer ara:

```
curl "http://127.0.0.1:8080/api/v1/customers/db/by-segment?segment=standard&limit=10"
```

Customer oluştur:

```
curl -X POST http://127.0.0.1:8080/api/v1/customers `
-H "Content-Type: application/json" `
-d '{"requestId":"req-1001","customerNo":"CUST-1001","fullName":"Ayşe Demir","segment":
"standard","email":"ayse.demir@example.com"}'
```

Segment güncelle:

```
curl -X PATCH http://127.0.0.1:8080/api/v1/customers/1/segment `
-H "Content-Type: application/json" `
-d '{"requestId":"req-1002","segment":"enterprise"}'
```

Status güncelle:

```
curl -X PATCH "http://127.0.0.1:8080/api/v1/customers/1/status/typed?status=passive&requestId=req-1003"
```

Customer sil:

```
curl -X DELETE http://127.0.0.1:8080/api/v1/customers/1 `
-H "Content-Type: application/json" `
-d "{\"requestId\":\"req-1004\",\"reason\":\"sample cleanup\"}"
```

Endpoint'ler

Endpoint	Amaç	Maliyet
GET /api/v1/catalog/nested	Provider hazır JSON döner.	En ucuz Dubbo read path.
GET /api/v1/customers/db	Provider DB'den liste okur ve JSON döner.	DB ve provider kapasitesine bağlıdır.
GET /api/v1/customers/db/{id}	Tek customer okur.	Küçük read.
POST /api/v1/customers	Customer oluşturur.	Write path, idempotency gerekir.
PATCH /api/v1/customers/{id}/segment	Segment günceller.	Aynı customer için key admission vardır.
DELETE /api/v1/customers/{id}	Customer siler.	Gerçek sistemde soft delete tercih edilebilir.

Profile Seçimi

Seçim	Ne zaman?	Ayarlar
micro-dubbo	Düşük RSS, kontrollü 503 kabul.	Küçük worker, queue ve connection.
micro-1x1 reçetesi	En küçük pod.	native-connections-per-endpoint=1, native-async-workers=1.
micro-2x2 reçetesi	Full sample DB write yapıyor veya Hikari 2 iken 1x1 çağrıları seri hale getiriyor.	sample.dubbo.capacity-profile=micro-2x2, connection 2, worker 2, queue 64, max-inflight 64, create route'ları ayrı ayrı 4/250 ms.
balanced-stable-4x4	Daha çok başarılı read RPS.	Connection ve worker 4, route budget kontrollü.

DB-backed endpoint için consumer ayarını client concurrency ile değil, provider Hikari kapasitesiyle başlatın.

Raw ve typed create endpoint'leri aynı provider command service'i kullanır. Route limitleri toplanır. Her iki route'u da 8 yapmak ortak 2x2 backend önünde 16 istek biriktirir. Aşağıdaki değerler PostgreSQL WAL commit gecikmesinin p99 boyunca büyümesini sınırlar:

```
reactor.rust.route-budget.rpc-customer-raw-create.route-admission.max-concurrent=4
reactor.rust.route-budget.rpc-customer-raw-create.route-admission.queue-timeout-ms=250
reactor.rust.route-budget.rpc-customer-typed-create.route-admission.max-concurrent=4
reactor.rust.route-budget.rpc-customer-typed-create.route-admission.queue-timeout-ms=250
```

Named budget aynı iş sınıfındaki route'lara ortak başlangıç değeri verir. Yalnızca bir endpoint farklı davranıyorsa `reactor.rust.route-admission.<route-key>.*` ile o endpoint'i override edin. Öncelik sırası annotation < workload < named budget < route-specific property şeklindedir.

DB Pool Ve c64/c256 Ne Demek?

c64 veya c256, load testte aynı anda kaç client request geldiğini gösterir.

Bu değer Hikari connection sayısı değildir.

Alan	Anlamı
c64	Aynı anda 64 client request baskısı.
reactor.dubbo.max-inflight	Consumer'ın aynı anda kaç RPC çağrısını uçuşta tutacağı.
route-admission.max-concurrent	Bir endpoint'in aynı anda kaç işi kabul edeceği.
sample.db.maximum-pool-size	Provider tarafındaki gerçek DB connection üst limiti.

Örnek:

```
c64 trafik gelir.
Provider Hikari pool 2 ise aynı anda sadece 2 DB işi çalışır.
Diğer işler kısa süre bekler veya kontrollü 503 alır.
```

DB-backed endpoint için doğru başlangıç:

```
reactor.dubbo.max-inflight=8
reactor.rust.route-budget.rpc-customer-raw-read.route-admission.max-concurrent=2
reactor.rust.route-budget.rpc-customer-raw-read.route-admission.queue-timeout-ms=75
```

Queue'yu büyötmek 503 oranını azaltabilir. Fakat RSS ve p99 değeri artırır.

Idle Connection Ve Provider Restart

Consumer, tamamlanmış Dubbo TCP connection'larını tekrar kullanır. Provider veya load balancer idle connection'ı kapatmış olabilir. Aşağıdaki ayar, uzun süre idle kalan socket'i yeni çağrıdan önce yeniler:

```
reactor.dubbo.native-idle-connection-ttl-ms=30000
```

Provider idle timeout değeri 20 saniye ise bu değeri örneğin 15000 yapın. Write çağrılarında `reactor.dubbo.retries=0` kalsın. Framework, kapalı idle socket'i request başlamadan atar; request byte'ları gönderildikten sonra duplicate işlem riski oluşturacak kör retry yapmaz. Restart testinde `nativeDubboStaleIdleConnectionsDiscarded` artmalı ve `nativeDubboErrors` sıfır kalmalıdır.

ZooKeeper Mi Static Service DNS Mi?

Seçim	Ne sağlar?	Ne zaman kullanılır?
Static Service DNS	En düşük consumer RSS. ZooKeeper client yoktur.	K8s Service zaten provider pod'larını load balance ediyorsa.
ZooKeeper	Provider register/re-register bilgisini takip eder.	Kurum standardı ZooKeeper ise veya registry zorunluysa.

Static örnek:

```
sample.dubbo.discovery=static
reactor.dubbo.providers=rest-sample-dubbo-provider:20880
```

ZooKeeper örnek:

```
sample.dubbo.discovery=zookeeper
reactor.dubbo.registry-address=zookeeper://zookeeper-client.platform.svc.cluster.local:2181
```

Sık Hatalar

Belirti	Muhtemel neden	Çözüm
503 artıyor	Route budget veya provider kapasitesi doldu.	Provider CPU, Hikari ve route metrics birlikte kontrol edilir.
p99 yükseliyor	Queue fazla büyüdü veya DB yavaşladı.	Queue timeout düşürün, provider DB wait ölçün.
Provider bulunamıyor	Static adres veya ZooKeeper config yanlış.	<code>reactor.dubbo.providers</code> veya registry adresini kontrol edin.
RSS yükseliyor	Full surface, typed DTO veya büyük queue kullanılıyor.	<code>native-static-consumer</code> , <code>micro-dubbo</code> ve daha küçük queue deneyin.